Smart Waste Management System: A Desktop Application with Mock Detection and Database-Driven Analytics

Muradul Islam

ID# 22203234

A Practicum in the Partial Fulfillment of the Requirements

for the Award of Bachelor of Computer Science and Engineering (BCSE)

Department of Computer Science and Engineering

College of Engineering and Technology

IUBAT—International University of Business Agriculture and Technology

Spring 2026

Smart Waste Management System: A Desktop Application with Mock Detection and Database-Driven Analytics

Muradul Islam

ID# 2203234

A Practicum in the Partial Fulfillment of the Requirements for the

Award of Bachelor of Computer Science and Engineering (BCSE)

The practicum has been examined and approved,

_____

Prof. Dr. Utpal Kanti Das

Chairman

Department of Computer Science and Engineering

_____

Shahinur Alam

Co-supervisor, Coordinator and Assistant Professor

Department of Computer Science and Engineering

_____

[Supervisor Name]

[Supervisor Designation]

Department of Computer Science and Engineering

Department of Computer Science and Engineering

College of Engineering and Technology

IUBAT—International University of Business Agriculture and Technology

Spring 2026

Letter of Transmittal

9 May 2026

The Chair
Practicum Defense Committee
Department of Computer Science and Engineering
IUBAT—International University of Business Agriculture and Technology
4 Embankment Drive Road, Sector 10, Uttara Model Town
Dhaka 1230, Bangladesh.

Subject: Submission of Practicum Report on the Smart Waste Management System.

Dear Sir,

With due respect, I am submitting my practicum report titled "Smart Waste Management System: A Desktop Application with Mock Detection and Database-Driven Analytics" as a partial requirement for the Bachelor of Computer Science and Engineering degree at IUBAT. The work covers the analysis, design, and implementation of a Python-based desktop application that supports authenticated waste detection workflows, role-based user management, alert configuration, analytics dashboards, and PDF report generation. A placeholder detection module has been integrated to allow end-to-end testing of the application before the YOLOv8 model is connected in a later phase.

Yours sincerely,

_____

Muradul Islam

ID# [Student ID]

Organization's Certificate

The scanned copy of the certificate issued by the host organization will be placed here.

[INSERT SCANNED ORGANIZATION CERTIFICATE]

Student's Declaration

I, Muradul Islam, declare that the work presented in this practicum report titled "Smart Waste Management System: A Desktop Application with Mock Detection and Database-Driven Analytics" is the result of my own effort. The analysis, design, code, and written discussion were prepared by me during the practicum period under the guidance of my supervisor and the co-supervisor named in the approval page. Where I have drawn on the work of others, I have cited the source in the body of the report and listed the full reference at the end.

I confirm that no part of this report contains plagiarised material or fabricated data. I take full responsibility for the accuracy of the work submitted, and I understand that any breach of academic integrity will be subject to the rules of the Department of Computer Science and Engineering, IUBAT.

_____

Muradul Islam

ID# [Student ID]

Supervisor's Certification

This is to certify that the practicum report titled "Smart Waste Management System: A Desktop Application with Mock Detection and Database-Driven Analytics" submitted by Muradul Islam, bearing ID# [Student ID], has been carried out under my supervision in partial fulfillment of the requirements for the award of Bachelor of Computer Science and Engineering at IUBAT—International University of Business Agriculture and Technology.

To the best of my knowledge, the work is original. The student has followed the standards expected of a final-year practicum and has shown the technical understanding, analytical reasoning, and writing quality required of the program. I recommend that the report be accepted for evaluation by the practicum defense committee.

_____

[Supervisor Name]

[Supervisor Designation]

Department of Computer Science and Engineering

IUBAT—International University of Business Agriculture and Technology

Abstract

Solid waste management in Bangladesh faces persistent operational gaps: missed collections, unsegregated waste streams, and limited visibility into the categories and volumes of waste generated by a facility on any given day. Manual logging is slow, error-prone, and difficult to audit. The Smart Waste Management System, abbreviated as SWMS, is a Windows desktop application built to address these gaps at the facility level. The system records every detection event in a structured database, enforces role-based access for administrators, supervisors, and operators, and gives each role a workspace tuned to the actions they are allowed to perform.

The work documented in this report covers the full software engineering lifecycle for the desktop application shell. Requirements were gathered from the project supervisor, from a review of similar systems in the literature, and from the practical needs of a small recycling operation. The system was designed using activity diagrams, swimlane diagrams, sequence diagrams, CRC cards, and a class diagram. Risks were identified and tracked using an RMMM plan, schedule was estimated using function points, and costs were summed across personnel, hardware, software, and other categories. The implementation uses Python 3.10 with PyQt5 for the user interface, SQLAlchemy on SQLite for persistence, bcrypt for password hashing, Matplotlib for embedded charts, and ReportLab for PDF generation. A placeholder detection engine was implemented so the application can be tested end-to-end before the YOLOv8 model is integrated in a later phase. The placeholder produces realistic category labels, confidence scores, and bin-fill levels, and writes annotated result images to disk.

Six representative test cases were executed against the build, covering authentication, mock detection, history filtering, alert triggering, report generation, and user management. All cases passed against their expected results. The remaining work, namely the integration of a trained computer vision model, a live camera feed, and scheduled reporting, has been deferred to a follow-up phase and is described in the limitations and future work sections. The deliverable is a stable, well-structured application that gives a small waste management operation a usable digital record from day one and a clear path to automated detection later.

Acknowledgments

All praise to Almighty Allah, who gave me the strength and patience needed to complete this practicum within the time available.

I am grateful to my supervisor, [Supervisor Name], for the careful reviews and the steady stream of practical advice that shaped the work. The pace of the project would not have been possible without that guidance. I also thank Shahinur Alam, the practicum coordinator, for his help with the administrative process and his prompt responses to every query I raised. Prof. Dr. Utpal Kanti Das, Chairman of the Department of Computer Science and Engineering, was generous with his time and attention, and I benefited from his suggestions during the milestone reviews.

I want to thank my classmates who joined informal discussions about the system design, particularly the early sessions on the alert rule engine and the report layout. Their questions made me clarify ideas I had taken for granted. Finally, I thank my family for their patience during the long evenings of writing and debugging. None of this work would have been finished without their support.

Muradul Islam

ID# 22203234

Table of Contents

Letter of Transmittal iii
Organization's Certificate iv
Student's Declaration v
Supervisor's Certification vi
Abstract vii
Acknowledgments viii
Table of Contents ix
List of Figures xiv
List of Tables xv

Chapter 1. Introduction 1
1.1 Background of the Study 2
1.2 Methodology 3
1.2.1 Primary Sources. 3
1.2.2 Secondary Sources. 4
1.3 Objectives 4
1.3.1 Broad Objective. 4
1.3.2 Specific Objectives. 5
1.4 Process Model 5
1.5 Feasibility Study 6
1.5.1 Technical Feasibility. 6
1.5.2 Economic Feasibility. 7
1.5.3 Operational Feasibility. 7
1.6 Structure of the Report 8

Chapter 2. Organizational Overview 9
2.1 Organization Vision 10
2.2 Organization Mission 10
2.3 Organization Services 10
2.4 Organizational Structure 11
2.5 My Position in this Organization 11
2.6 Address of the Organization 12

Chapter 3. Requirement Engineering 13

3.1 Requirement Engineering Process 14
3.2 Requirement Elicitation 14
3.3 Requirement Analysis 15
3.4 Requirement Specifications 16
3.4.1 User Requirements. 16
3.4.2 System Requirements. 17
3.4.3 Functional Requirements. 19
3.4.4 Non-Functional Requirements. 20
3.5 Use Case Diagram 21

Chapter 4. Analysis Modeling 22
4.1 Activity Diagrams 23
4.2 Swimlane Diagrams 27
4.3 Sequence Diagrams 30
4.4 CRC Cards 32
4.5 Class Diagram 37

Chapter 5. Risk Management 38
5.1 Risk Management Overview 39
5.2 Risk Identification 39
5.3 Risk Analysis 40
5.4 Risk Planning 41
5.4.1 RMMM Planning. 41

Chapter 6. Project Planning and Scheduling 47
6.1 Overview 48
6.2 Function Point Estimation 48
6.2.1 Functionality, Input, and Output. 48
6.2.2 Identify Complexity. 49
6.2.3 UFP Contribution. 50
6.2.4 TDI Calculation. 52
6.2.5 Final Calculation. 53
6.3 Project Scheduling 53

Chapter 7. Project Cost Estimation 55
7.1 Overview 56

7.2 Personnel Cost 56
7.3 Hardware Cost 57
7.4 Software Cost.....58
7.5 Other Cost 58
7.6 Accounts Table 59

Chapter 8. Designing 60
8.1 Data Flow Diagrams 61
8.2 ER Diagram 67
8.3 Database Field Design 68
8.4 Interface Design 69

Chapter 9. Testing.....78
9.1 Test Cases 79

Chapter 10. Ethical Consideration and Sustainability 85
10.1 Ethical Considerations 86
10.2 Sustainability Through Software Engineering.....87

Chapter 11. Conclusion.....89
11.1 Conclusion 90
11.2 Limitation.....90
11.3 Future Plan 91

References.....92
Plagiarism Report.....94

List of Figures

Figure 1.1 Iterative-Incremental Process Model.....5
Figure 2.1 Organizational Structure.....11
Figure 3.1 Use Case Diagram of SWMS.....21
Figure 4.1 Activity Diagram: User Authentication.....23
Figure 4.2 Activity Diagram: Waste Detection.....24
Figure 4.3 Activity Diagram: Detection Verification.....25
Figure 4.4 Activity Diagram: Report Generation.....26
Figure 4.5 Swimlane Diagram: End-to-End Detection Workflow.....27
Figure 4.6 Swimlane Diagram: Alert Rule Lifecycle.....28
Figure 4.7 Swimlane Diagram: User Management.....29
Figure 4.8 Sequence Diagram: Detection Process.....30
Figure 4.9 Sequence Diagram: Alert Trigger Flow.....31
Figure 4.10 Class Diagram of SWMS.....37
Figure 6.1 Gantt Chart of Project Schedule.....54
Figure 8.1 DFD Level 0 (Context Diagram).....61
Figure 8.2 DFD Level 1.....62
Figure 8.3 DFD Level 2: Detection Process.....63
Figure 8.4 DFD Level 2: Alert Process.....64
Figure 8.5 DFD Level 2: Report Generation.....65
Figure 8.6 ER Diagram.....67
Figure 8.7 Database Field Design.....68
Figure 8.8 Login Screen Interface.....70
Figure 8.9 Dashboard Interface.....71
Figure 8.10 Detection Screen Interface.....72
Figure 8.11 History Screen Interface.....73
Figure 8.12 Users Screen Interface.....74
Figure 8.13 Alerts Screen Interface.....75
Figure 8.14 Reports Screen Interface.....76
Figure 8.15 Settings Screen Interface.....77

List of Tables

Table 3.1 User Requirements....16
Table 3.2 Functional Requirements....19
Table 3.3 Non-Functional Requirements....20
Table 4.1 CRC Card: User....32
Table 4.2 CRC Card: Detection....33
Table 4.3 CRC Card: AlertRule....33
Table 4.4 CRC Card: Alert....34
Table 4.5 CRC Card: Report....35
Table 4.6 CRC Card: ActivityLog....35
Table 4.7 CRC Card: AppSetting....36
Table 5.1 Risk Identification....39
Table 5.2 Risk Analysis....40
Table 5.3 Risk Planning....41
Table 5.4 RMMM: Schedule Risk....42
Table 5.5 RMMM: Technical Risk (Model Integration)....43
Table 5.6 RMMM: Data Loss Risk....43
Table 5.7 RMMM: Security Risk....44
Table 5.8 RMMM: Performance Risk....45
Table 5.9 RMMM: Usability Risk....45
Table 5.10 RMMM: Operational Risk (SMTP Outage)....46
Table 6.1 System Functionalities, Inputs, and Outputs....48
Table 6.2 Complexity Weights for External Inputs....49
Table 6.3 Complexity Weights for External Outputs....49
Table 6.4 UFP Contribution: External Inputs....50
Table 6.5 UFP Contribution: External Outputs....50
Table 6.6 UFP Contribution: External Inquiries....51
Table 6.7 UFP Contribution: Internal Logical Files....51
Table 6.8 UFP Contribution: External Interface Files....52
Table 6.9 UFP Total Summary....52
Table 6.10 TDI General System Characteristics....52
Table 6.11 Final Function Point Calculation....53
Table 7.1 Working Day Calculation....56
Table 7.2 Personnel Cost....57

Table 7.3 Hardware Cost 57
Table 7.4 Software Cost.....58
Table 7.5 Other Cost 58
Table 7.6 Accounts Table 59
Table 9.1 Test Case: User Login.....79
Table 9.2 Test Case: Run Mock Detection 80
Table 9.3 Test Case: History Filtering and Export 81
Table 9.4 Test Case: Alert Rule and Trigger 82
Table 9.5 Test Case: Report Generation 83
Table 9.6 Test Case: User Management (Admin) 84

Chapter 1.
Introduction

Solid waste handling is one of the more difficult day-to-day operations for small and medium recycling facilities in Bangladesh. Waste arrives in mixed batches, segregation is done by hand, and the records of how much of each category passed through a site on a given day are usually written on paper or kept in spreadsheets that nobody opens after the shift ends. This practicum was undertaken to address the recordkeeping and supervision side of that problem with a desktop application that an operator can use without an internet connection, that a supervisor can audit, and that an administrator can configure for the specific waste categories handled by their site. The application is named the Smart Waste Management System, abbreviated as SWMS in the rest of this report. The current version is a desktop shell with a placeholder detection engine. The detection engine produces realistic looking results so that every other part of the system can be developed and tested before a trained YOLOv8 model is connected in the next phase of the project.

1.1 Background of the Study

Bangladesh generates more than twenty thousand tonnes of municipal solid waste each day according to the Department of Environment, and a sizeable share of that flow is handled by small private operators who buy recyclables from collectors, sort the material on site, and pass it to industrial recyclers. These operators rarely have any digital tooling. The supervisor walks through the sorting area with a notebook, jots down approximate counts of bins filled with each category of waste, and sends the figures to the office at the end of the day. This causes three operational problems that came up repeatedly in the conversations I had while scoping the practicum.

The first problem is delayed visibility. Because counts are entered into spreadsheets at the end of the shift, supervisors only learn about a sudden spike in plastic waste, or an unusual run of e-waste, hours after it happened. By that time the bins have been emptied and the evidence is gone. The second problem is the absence of an audit trail. When a customer disputes the weight or category of a delivered batch, the operator has no easy way to retrieve the record because the paper logs are filed by date and not by customer or category. The third problem is alerting. Bins fill up,

sometimes overflow, and the team only notices when the smell or the spillage becomes obvious. There is no rule-based mechanism that says "if more than thirty plastic detections occur in a single day, send an email to the supervisor". The literature on waste management informatics, summarised by Rahman et al. (2022) and by the World Bank's What a Waste 2.0 dataset, identifies the same gaps in similar economies and recommends low-cost digital recordkeeping with structured alerts as a starting point.

The Smart Waste Management System was scoped as a response to those three problems. The desktop form factor was chosen so that the system can run on the inexpensive Windows machines that are already common in office back rooms in Dhaka. The local SQLite database keeps the system running when the internet is unavailable. A placeholder detection engine was built first because it allows the rest of the application — the screens, the alert engine, the report generator, and the user management to be developed, tested, and demonstrated without waiting for the model. When the model is ready, only one file in the codebase needs to change.

1.2 Methodology

Two categories of data sources were used to gather information for the practicum. They are described below.

1.2.1 Primary Sources.

Primary information was collected through the following channels:

- Weekly meetings with the practicum supervisor, where the scope of each phase was agreed and the deliverables were reviewed against the build document.
- An informal interview with the manager of a recycling collection point in Uttara, who described the daily workflow, the waste categories typically handled, and the reasons paper logs are still used.
- Two on-site observations of the sorting workflow, used to confirm the role hierarchy (administrator, supervisor, operator) and the separation of duties between roles.
- Hands-on testing of every screen and every workflow during development, with notes recorded in a development journal that was reviewed at the end of each phase.

1.2.2 Secondary Sources.

Secondary information was drawn from the following:

- Academic papers on waste classification using deep learning, including Yang and Thung (2017) on the TrashNet dataset and recent work on YOLO-based detection by Adedeji and Wang (2019).
- Government and inter-agency reports, including the Department of Environment 2021 status report and the World Bank What a Waste 2.0 dataset.
- Official documentation for PyQt5, SQLAlchemy, ReportLab, and Matplotlib, which guided the implementation choices.
- Public engineering blog posts on bcrypt password hashing and on PyQt threading patterns.

1.3 Objectives

The objectives of the proposed system are split into a single broad objective and a set of specific objectives that translate the broad statement into measurable deliverables.

1.3.1 Broad Objective.

The broad objective of the practicum is to build a stable Windows desktop application that allows a small recycling facility to record, classify, and audit waste detection events; to generate periodic PDF reports for the management; and to receive automated alerts when waste counts breach configured thresholds, with a clean interface for replacing the placeholder detection engine with a trained YOLOv8 model in a later phase.

1.3.2 Specific Objectives.

The specific objectives of the project are listed below:

- Design and implement a relational schema in SQLite using SQLAlchemy ORM that stores users, detections, alert rules, alerts, activity logs, reports, and application settings.
- Implement role-based access control with three roles (administrator, supervisor, operator) and enforce the role rules across every screen of the application.
- Build a placeholder detection engine that returns realistic category labels, confidence scores, and bin-fill levels and that writes annotated result images to disk.

- Build a dashboard, a detection screen, a history screen with filters and export, an alerts screen, a reports screen, a users screen for the administrator, and a settings screen with SMTP configuration.
- Generate PDF reports through ReportLab containing summary statistics, charts rendered with Matplotlib, and a paginated table of detections.
- Provide an alert engine that evaluates configurable thresholds against the detection log and dispatches email notifications through the configured SMTP server.

1.4 Process Model

The iterative-incremental process model was chosen for the practicum. The system is large enough that a single waterfall run would have left integration risks until the very end, and small enough that a heavyweight Scrum setup with a separate team would have added overhead without benefit. The iterative-incremental model fits because each phase produces a working increment that the supervisor can review before the next phase begins. The phases used were Foundation, Core Logic, UI Widgets, UI Screens, Entry Point and Integration, and Polish, in the order shown in the build document. Each iteration repeated the same internal cycle of analysis, design, implementation, and verification, with a checkpoint review at the boundary between phases.

The reasons for choosing this model are listed below:

- It exposes integration problems early because every phase ends with a runnable system, which means the database, the engines, and the UI are exercised together from very early in the project.
- It accommodates scope changes inside a phase without disturbing the overall plan, which matters for a student project where the scope tends to drift as new ideas surface.
- It produces a partially working application after the first two phases, which gives the supervisor something to review long before the project is complete.
- It supports the deferred model integration cleanly: the placeholder is a complete increment that satisfies all upstream expectations, so the model integration phase can be planned and resourced as a separate iteration without rework.

- It maps neatly onto the build document, which is structured as an ordered list of phases with explicit verification steps.

Figure 1.1 Iterative-Incremental Process Model.

Figure 1.1 illustrates the cycle. Each phase moves through the four activities clockwise and produces a tested increment that becomes the input to the next phase.

1.5 Feasibility Study

Three feasibility dimensions were considered before development started. The conclusions for each dimension are summarised below.

1.5.1 Technical Feasibility.

All chosen technologies are open source, well documented, and supported on Windows 10 and Windows 11. PyQt5 has been the standard cross-platform desktop GUI framework in Python for more than a decade. SQLAlchemy is the most widely used ORM in the Python ecosystem and SQLite is the default zero-configuration database for desktop applications. bcrypt, Matplotlib, and ReportLab have stable Python bindings and large user bases. The placeholder detection engine uses Pillow only, which is preinstalled on most Python distributions. The hardware requirement (a 4 GB RAM Windows machine) matches the office machines already in use at the target facility. None of the components require a paid licence or a network connection during normal use, so there is no technical barrier to deployment.

1.5.2 Economic Feasibility.

The total cost of the project is dominated by personnel time. The hardware was already available, the software stack is open source, and the only ongoing cost is the SMTP service for outbound alerts, which can be the supervisor's existing Gmail account at no additional charge. The detailed breakdown is presented in Chapter 7. The figures show that the development cost falls within the budget the host organization is willing to absorb for a system that replaces paper logs and reduces the time the supervisor spends preparing weekly summaries by hand.

1.5.3 Operational Feasibility.

The application uses the same role hierarchy that already exists at the facility. Operators continue to do detection-time entry, supervisors continue to verify and approve, and administrators continue to manage users and settings. The interface uses Bengali-friendly Latin characters and a sidebar layout that matches the conventions of common business applications, so the learning curve for operators is short. A short user guide and a session-based training plan are sufficient to roll the system out, and the rollback plan in case of failure is to return to the existing paper logs without data loss. There is no operational dependency that would prevent adoption.

1.6 Structure of the Report

The report is organised into eleven chapters. Chapter 1 is this introduction. Chapter 2 covers the host organization. Chapter 3 covers requirement engineering, including elicitation, analysis, specification, and the use case diagram. Chapter 4 presents the analysis models: activity diagrams, swimlane diagrams, sequence diagrams, CRC cards, and the class diagram. Chapter 5 documents the risk management plan and the RMMM tables. Chapter 6 contains the function-point estimation and the project schedule. Chapter 7 estimates the cost. Chapter 8 covers the system design, including data flow diagrams, the ER diagram, the database field design, and the interface designs. Chapter 9 presents the test cases. Chapter 10 discusses ethics and sustainability. Chapter 11 concludes the report and lists the limitations and the future work. The references and the plagiarism report follow at the end.

Chapter 2.

Organizational Overview

This chapter introduces the host organization for the practicum, describes its vision and mission, lists the services it provides, presents its organizational structure, identifies my position during the practicum period, and gives the address of its office.

The practicum was carried out at GreenLoop Recycling Solutions, a Dhaka-based company that operates a sorting and aggregation facility for post-consumer recyclables and that supplies industrial customers with sorted plastic, metal, paper, and glass. The choice of host was driven by the supervisor's existing relationship with the company and by the company's interest in moving away from paper-based recordkeeping.

2.1 Organization Vision

The vision of GreenLoop Recycling Solutions is to be a recognised partner of choice for urban recyclables aggregation in Bangladesh, with measurable improvements in the volume of waste diverted from landfill and in the welfare of the workers handling that waste. The company sees itself as a bridge between informal collectors and the formal recycling industry, and it wants the bridge to be transparent, auditable, and safe.

2.2 Organization Mission

The mission of GreenLoop is to receive mixed recyclables, separate them into seven core categories (Plastic, Metal, Glass, Organic, Paper, Hazardous, and E-Waste), and supply the sorted material to industrial buyers at a fair price while keeping the operation compliant with the Department of Environment's handling guidelines. Digitising the recordkeeping is one of the steps needed to support this mission, because audits, customer disputes, and regulatory reporting all become easier when the underlying data is structured.

2.3 Organization Services

The services offered by GreenLoop are listed below:

a. Door-to-door pickup of mixed recyclables from registered households and small businesses in selected zones of Dhaka.

b. On-site sorting of received material into seven categories with weight measurement and digital recordkeeping.

c. Bulk supply of sorted material to industrial recyclers under monthly contracts.

d. Hazardous waste handling under licence, including segregation, packaging, and handover to authorised disposers.

e. Educational sessions for schools and resident associations on source segregation and safe waste handling.

f. Custom monthly reports to corporate clients who track their own diverted volumes for sustainability reporting.

2.4 Organizational Structure

The organization runs with a flat structure suited to a small team. The Managing Director sits at the top. Three department heads report to the Managing Director: Operations, Administration, and Business Development. The Operations head supervises the sorting team leaders, who in turn supervise the operators on each shift. The IT team sits inside the Administration department and is the team I joined for the practicum. The structure is shown in Figure 2.1.

Figure 2.1 Organizational Structure.

The diagram makes the reporting lines explicit. The IT team is small (three members during the practicum period) and is responsible for the company's website, the internal spreadsheets, and now the SWMS application.

2.5 My Position in this Organization

I joined the IT team as a Software Engineering Intern for the practicum period. My role was to design and develop the SWMS desktop application end-to-end, including the database, the engines, the user interface, and the test cases. I reported daily to the IT team lead and weekly to the

practicum supervisor at IUBAT. The role gave me direct access to the operations team for requirements gathering and access to the existing paper logs for sample data.

2.6 Address of the Organization

GreenLoop Recycling Solutions

House 27, Road 11, Sector 6

Uttara Model Town, Dhaka 1230

Bangladesh

Web: www.greenloop.com.bd (Placeholder address; the company name is illustrative and may be replaced by the actual host organization in the final submission.)

Chapter 3.
Requirement Engineering

Requirement engineering is the work of finding out what the system has to do, writing it down in a form that supports design, and confirming it with the people whose problems are being solved. This chapter walks through the four steps that produced the requirement set for SWMS: a brief framing of the process, the elicitation methods that fed the inputs, the analysis that grouped the raw input into themes, and the specification that turned the themes into testable statements. The chapter ends with the use case diagram for the system.

3.1 Requirement Engineering Process

The requirement engineering activity for SWMS followed three steps in sequence. Each step produced an artefact that was reviewed with the practicum supervisor before the next step began.

a. Elicitation: collect raw input from the host organization, the supervisor, and the literature review.

b. Analysis: organize the raw input into user, system, functional, and non-functional groups, resolve duplicates, and flag conflicts.

c. Specification: write each requirement as a numbered, testable statement and capture the use case diagram.

3.2 Requirement Elicitation

Elicitation began with an unstructured interview at the host organization. The IT team lead and the operations supervisor joined the session. The aim was to capture the daily workflow, the existing pain points, and the expectations the team had of a digital replacement for paper logs. The session produced about two pages of notes covering authentication, recordkeeping, alerts, and reporting. A second session was held a week later to confirm the role hierarchy and to discuss the workflow for verifying detection results, which is something the supervisor wanted to keep manual for the first deployment.

In parallel, the literature review surfaced the standard non-functional concerns for desktop applications used in industrial environments: response time, password handling, error recovery,

and offline operation. These were added to the elicitation backlog. Finally, the practicum supervisor reviewed the backlog and added two requirements that came from the academic side of the project, namely the requirement to provide CSV and Excel exports for the data and the requirement to keep the codebase organised for the model integration that comes in the next phase.

3.3 Requirement Analysis

The raw input from the elicitation step was sorted into the following themes.

1. Authentication and access control. The system has to identify the operator at every action, has to enforce different permission sets for the three roles, and has to keep passwords safe at rest.
2. Detection and recordkeeping. Every detection event has to be saved with a timestamp, the operator who ran it, the result image, the categories detected, the confidence scores, and an indicator of how full the bin was.
3. Verification. Supervisors review pending detections, confirm or reject them, and may add notes. The verification status is part of the audit trail.
4. Alerts and notifications. Administrators define rules with thresholds and periods. The system evaluates rules after each detection and sends an email when a threshold is breached.
5. Analytics and reporting. The dashboard summarises today's activity. The report engine generates PDF reports for daily, weekly, monthly, and custom ranges.
6. User management. Administrators add operators and supervisors, edit them, and deactivate them. Self-deactivation is not allowed.
7. Settings and audit. SMTP, company name, and detection threshold are configurable. Every important action writes to an activity log that can be queried later.
8. Operational quality. Long-running tasks must not freeze the user interface. Errors must be reported clearly. The application must work without internet access for everything except the email dispatch.

The themes were reviewed with the supervisor and validated. They became the headings for the specification step that follows.

3.4 Requirement Specifications

3.4.1 User Requirements.

User requirements are written from the perspective of the people who use the system. They are listed in Table 3.1.

Table 3.1 User Requirements.

UR#	Description	Primary Role
UR-01	I can log in with a username and password.	All
UR-02	I can change my password from the settings screen.	All
UR-03	I can upload an image and run waste detection on it.	Operator
UR-04	I can see my own detection history with the original and annotated images.	Operator
UR-05	I can see all detection records, filter by date, category, and status.	Supervisor, Admin
UR-06	I can verify or reject any pending detection and add a note.	Supervisor, Admin
UR-07	I can export the detection list as CSV or Excel for offline analysis.	Supervisor, Admin
UR-08	I can view a dashboard with today's totals, the most common waste type, and active alerts.	All
UR-09	I can generate a PDF report for a daily, weekly, monthly, or custom date range.	Supervisor, Admin
UR-10	I can configure alert rules with a category, a threshold, a period, and a notification email.	Admin
UR-11	I can acknowledge alerts so the team knows they are being handled.	Supervisor, Admin
UR-12	I can manage user accounts: add, edit, deactivate, and reactivate.	Admin

UR#	Description	Primary Role
UR-13	I can configure the SMTP settings for outbound email.	Admin
UR-14	I can view the activity log to see who did what and when.	Admin
UR-15	I can take a backup of the database file from the settings screen.	Admin

Table 3.1 lists the fifteen user-facing capabilities expected from the system, with the role that owns each capability.

3.4.2 System Requirements.

System requirements describe the modules of the application and the actions each module supports. The list below is organised by module and follows the build document of the project.

9. Authentication module: hash passwords with bcrypt; verify credentials at login; track last login timestamp; expose a check_permission helper that respects the role hierarchy admin > supervisor > operator.
10. User management module: create, update, deactivate, and list users; prevent self-deactivation; emit an activity log entry for every change.
11. Detection engine: accept an image path and a user ID; produce one to four random detections from the seven configured categories; assign confidence scores between 0.65 and 0.98; assign a bin-fill level; draw bounding boxes on a copy of the image and save the annotated image to the results folder; save a detection row to the database.
12. Detection history module: query detections with filters for date range, category, status, and operator; paginate results at twenty-five rows per page; expose CSV and Excel exporters.
13. Verification workflow: allow a supervisor or administrator to mark a pending detection as verified or rejected; record the verifier and the timestamp; allow notes to be appended.
14. Analytics engine: compute totals for today and for all time; compute category distribution and fill-level distribution for a given date range; compute daily counts for a chart period; compute operator performance figures.

15. Report engine: generate a PDF for the requested period; embed a category chart rendered with Matplotlib; include summary statistics, a fill-level summary, and a paginated detection table; save the PDF and a row in the reports table.
16. Alert rules module: create, update, delete, and list alert rules with category, threshold, period, notification email, and active flag.
17. Alert engine: evaluate every active rule after each detection; create an alert row when the threshold is breached; route the alert to the notification service when an email address is set on the rule; expose acknowledgement and listing.
18. Notification service: read SMTP settings from the application settings table; format and send emails through smtplib; report success or failure to the alert engine.
19. Activity logging module: write an entry whenever a user logs in, runs a detection, verifies a detection, edits or deactivates a user, creates or updates an alert rule, acknowledges an alert, generates a report, or changes a setting.
20. Settings module: store key-value settings in the database; expose a settings screen for SMTP, company name, default confidence threshold, and a database backup action.
21. UI shell: a left sidebar with role-aware navigation buttons; a stacked widget on the right; a logout action; a toast notification system; loading indicators for long actions.
22. Concurrency: every action that writes to the database or the file system runs in a QThread so the UI thread does not block.
23. Persistence: SQLite database file at database/waste_management.db; default admin user seeded on first run; default settings seeded on first run.

3.4.3 Functional Requirements.

The functional requirements are listed in Table 3.2 with one row per requirement.

Table 3.2 Functional Requirements.

FR#	Description	Priority
FR-01	The system shall require a username and password to grant access.	High
FR-02	The system shall hash passwords with bcrypt and store only the hash.	High
FR-03	The system shall update last_login on every successful authentication.	Medium

FR#	Description	Priority
FR-04	The system shall allow an operator to upload an image of file type PNG, JPG, or JPEG and run a detection on it.	High
FR-05	The system shall save every detection event with image path, result image path, waste category, confidence, fill level, operator, status, and timestamp.	High
FR-06	The system shall allow supervisors and administrators to verify or reject any pending detection.	High
FR-07	The system shall allow supervisors and administrators to filter detections by date range, category, and status, and to export the filtered list.	High
FR-08	The system shall display a dashboard with today's detection count, the most common category, and the active alert count.	Medium
FR-09	The system shall render a category distribution chart and a daily-count chart for the configured period.	Medium
FR-10	The system shall generate a PDF report for daily, weekly, monthly, or custom date ranges.	High
FR-11	The system shall allow administrators to define alert rules with category, threshold, period, and notification email.	High
FR-12	The system shall evaluate active alert rules after each detection and create alert rows when thresholds are breached.	High
FR-13	The system shall send an email through the configured SMTP server when a rule with a notification email is breached.	Medium
FR-14	The system shall allow supervisors and administrators to acknowledge alerts.	Medium
FR-15	The system shall allow administrators to add, edit, deactivate, and reactivate users, but never to deactivate themselves.	High
FR-16	The system shall write an activity log entry for every privileged action.	Medium

Table 3.2 lists the sixteen functional requirements that drive the implementation.

3.4.4 Non-Functional Requirements.

Non-functional requirements describe how the system has to behave rather than what it has to do. They are listed in Table 3.3.

Table 3.3 Non-Functional Requirements.

Category	Requirement	Target
Performance	A mock detection on a 1080p image shall complete within 2 seconds on a 4 GB RAM Windows laptop.	< 2 s
Performance	A PDF report covering a one-month period with up to 1,000 detections shall generate within 10 seconds.	< 10 s
Usability	An operator shall be able to run a detection from a cold start in under five clicks.	5 clicks
Usability	All destructive actions shall require an explicit confirmation dialog.	Required
Reliability	A failure during PDF generation shall leave the database in a consistent state.	ACID
Security	Passwords shall never be transmitted or stored in plain text.	bcrypt
Security	SMTP credentials shall be stored in the application settings table and never written to logs.	Required
Maintainability	The mock detection engine shall expose the same method signature as the planned model engine, so the model integration only changes one file.	Required
Maintainability	The codebase shall keep core, database, and ui layers in separate Python packages.	Required
Portability	The application shall run on Windows 10 and Windows 11 without modification.	Required
Offline	The application shall function without internet access for every action except sending email.	Required
Auditability	Every privileged action shall be retrievable from the activity log within 30 days.	Required

Table 3.3 lists the non-functional requirements grouped by category, with the target for each.

3.5 Use Case Diagram

The use case diagram for SWMS is shown in Figure 3.1. Three actors interact with the system: the Administrator, the Supervisor, and the Operator. The use cases are drawn so the inheritance of permissions is visible: every supervisor case is also available to the administrator, and every operator case is available to all three roles.

Figure 3.1 Use Case Diagram of SWMS.

The diagram makes it easy to see that an operator's surface is small (login, run detection, and view their own history), the supervisor's surface adds verification and reporting, and the administrator's surface covers everything including user and settings management. The diagram was the source of truth for the role-based checks scattered across the screens, and any deviation found during testing was fixed by aligning the code with the diagram.

Chapter 4.

Analysis Modeling

Analysis modeling moves the requirements from a list of statements to a set of diagrams that describe the dynamic and the static structure of the system. This chapter contains four diagram families used in SWMS: activity diagrams that show the flow of work across each major use case, swimlane diagrams that map activities onto the actors who perform them, sequence diagrams that show the messages between objects, and CRC cards that capture the responsibilities of each class. The chapter ends with the class diagram that summarises the static structure.

4.1 Activity Diagrams

Activity diagrams were drawn for the four most important use cases of the system. Each diagram starts at a solid black circle, ends at a circled solid circle, and shows the decision points as diamonds. The diagrams are introduced one by one in the figures below.

Figure 4.1 Activity Diagram: User Authentication.

Figure 4.1 shows the user authentication flow. The decision diamond is the only branching point. A failed login does not advance the user and is logged silently to avoid leaking information about the existence of usernames.

Figure 4.2 Activity Diagram: Waste Detection.

Figure 4.2 shows the detection flow. The placeholder engine takes the place of a real model but produces the same shape of output, so every downstream activity (alert evaluation, toast notification, history update) runs identically when the model is plugged in later.

Figure 4.3 Activity Diagram: Detection Verification.

Figure 4.3 shows the verification flow used by supervisors and administrators. The activity is one of the few places where different downstream behaviour depends on the user's choice, and the diagram makes that branching explicit.

Figure 4.4 Activity Diagram: Report Generation.

Figure 4.4 shows the report generation flow. The QThread step is included because the report build can take several seconds for a long date range and would otherwise freeze the user interface.

4.2 Swimlane Diagrams

Swimlane diagrams partition activities into vertical lanes, one per actor or component. They are useful when the same activity is performed differently by different roles. Three swimlane diagrams were drawn.

.

Figure 4.5 Swimlane Diagram: End-to-End Detection Workflow.

Figure 4.5 shows the operator workflow with the three downstream components rendered as separate lanes. The diagram clarifies which component owns which step and was used during code review to confirm that responsibilities were not leaking across module boundaries.

Figure 4.6 Swimlane Diagram: Alert Rule Lifecycle.

Figure 4.6 shows the alert rule lifecycle. The lifecycle starts when an administrator creates the rule and continues every time a detection is recorded. The Notification Service is shown as a distinct lane because its failure mode (SMTP outage) must not break the alert flow.

Figure 4.7 Swimlane Diagram: User Management.

Figure 4.7 shows the user management swimlane. Note the explicit guard against self-deactivation, which is a real bug class encountered in similar systems and is captured as a discrete decision step.

4.3 Sequence Diagrams

Sequence diagrams capture the timing of messages between collaborating objects. Two sequence diagrams were produced for the most message-rich workflows.

Figure 4.8 Sequence Diagram: Detection Process.

Figure 4.8 shows the messages exchanged during a detection. The diagram encodes one critical contract: the alert evaluation runs after the detection has been committed, so a partially saved detection cannot trigger a phantom alert.

Figure 4.9 Sequence Diagram: Alert Trigger Flow.

Figure 4.9 shows the alert trigger flow. The optional fragments make the conditional behaviour explicit, and the SMTP error path returns False without raising, so the calling code never crashes because of a transient mail outage.

4.4 CRC Cards

CRC cards (Class, Responsibility, Collaborator) are a lightweight technique for recording the responsibilities of each class and the other classes it relies on. Seven CRC cards were produced for the SWMS classes.

Table 4.1 CRC Card: User.

Class Name	User
Class Type	Domain Entity
Class Characteristics	Concrete; identifiable; persisted
Responsibilities	- Hold the credentials, role, and activation state of a person who interacts with SWMS. - Provide a stable identifier for foreign keys in detections, alerts, reports, and activity logs. - Track last login timestamp and account creation timestamp.
Collaborators	- AuthManager (validates credentials and toggles active state). - Detection (records the operator and verifier). - Alert (records the acknowledger). - ActivityLog (records the actor).

Table 4.1 records the responsibilities of the User class.

Table 4.2 CRC Card: Detection.

Class Name	Detection
Class Type	Domain Entity
Class Characteristics	Concrete; identifiable; persisted; produced by the detection engine
Responsibilities	- Hold the original image path, the annotated result image path, the predicted waste category, the confidence score, and the bin fill level. - Record who performed the detection and who later verified or rejected it. - Carry the verification status that gates downstream reporting.
Collaborators	- DetectionEngine (creator). - User (operator and verifier). - AnalyticsEngine (reads aggregate counts). - ReportEngine (lists detections in PDFs).

Table 4.2 records the responsibilities of the Detection class.

Table 4.3 CRC Card: AlertRule.

Class Name	AlertRule
Class Type	Domain Entity
Class Characteristics	Concrete; identifiable; persisted; configurable by administrators
Responsibilities	- Capture the category, threshold, period, and notification email for a single alert rule. - Carry an activation flag so rules can be turned off without deletion. - Record who created the rule and when.
Collaborators	- AlertManager (evaluator). - Alert (children produced when the rule fires). - User (creator).

Table 4.3 records the responsibilities of the AlertRule class.

Table 4.4 CRC Card: Alert.

Class Name	Alert
Class Type	Domain Entity
Class Characteristics	Concrete; identifiable; persisted; created by the alert engine
Responsibilities	- Hold the message, severity, and triggered timestamp for a fired alert. - Track acknowledgement status and the acknowledging user. - Reference the rule that produced it.
Collaborators	- AlertRule (parent). - AlertManager (creator). - NotificationService (consumer for email dispatch). - User (acknowledger).

Table 4.4 records the responsibilities of the Alert class.

Table 4.5 CRC Card: Report.

Class Name	Report
Class Type	Domain Entity
Class Characteristics	Concrete; identifiable; persisted; produced by the report engine

Responsibilities	- Capture the report type, the date range, and the file path of the generated PDF. - Record the user who generated the report and the generation timestamp.
Collaborators	- ReportEngine (creator). - User (generator). - AnalyticsEngine (data source).

Table 4.5 records the responsibilities of the Report class.

Table 4.6 CRC Card: ActivityLog.

Class Name	ActivityLog
Class Type	Domain Entity
Class Characteristics	Concrete; identifiable; persisted; append-only
Responsibilities	- Record a single audit event with actor, action, optional details, and timestamp. - Provide a stable history that can be queried by user, by action, and by date range.
Collaborators	- LogManager (writer and reader). - User (actor). - All modules that perform privileged actions (callers).

Table 4.6 records the responsibilities of the ActivityLog class.

Table 4.7 CRC Card: AppSetting.

Class Name	AppSetting
Class Type	Domain Entity
Class Characteristics	Concrete; identifiable by key; persisted; mutable by administrators
Responsibilities	- Store named configuration values for SMTP, company branding, and detection threshold. - Allow values to be read and written without disturbing the schema.
Collaborators	- SettingsScreen (editor).

	- NotificationService (consumer of SMTP keys). - DetectionEngine (consumer of threshold).

Table 4.7 records the responsibilities of the AppSetting class.

4.5 Class Diagram

The class diagram in Figure 4.10 brings the seven entities together with their attributes, methods, and relationships. The relationships are drawn with the standard UML notation: a solid line for association, a hollow diamond for aggregation, and the multiplicity at each end.

Figure 4.10 Class Diagram of SWMS.

Figure 4.10 is the static counterpart to the sequence and activity diagrams above. It is the diagram that maps directly onto the SQLAlchemy models in database/models.py and was used to verify that every documented relationship has a matching ForeignKey and back-populates pair in the code.

Chapter 5.
Risk Management

Risk management is the discipline that lets a project anticipate problems before they occur, plan a response in advance, and react with a known playbook when a problem surfaces. The practicum followed the four standard stages of risk management: identification, analysis, planning, and an RMMM (Risk Mitigation, Monitoring, and Management) plan for each non-trivial risk. The chapter walks through each stage in turn.

5.1 Risk Management Overview

Risks for SWMS were grouped into the following categories. Each category contains one or more concrete risks documented in the tables that follow.

a. Schedule risk: the chance that a phase finishes later than planned and pushes the next phase into less time than it needs.

b. Technical risk: the chance that a chosen technology does not behave as expected, with the model integration step being the most exposed area.

c. Data risk: the chance of losing detection records, result images, or generated reports due to disk failure or accidental deletion.

d. Security risk: the chance of credentials, SMTP secrets, or audit logs being exposed.

e. Performance risk: the chance that the application becomes slow with a year of recorded data.

f. Usability risk: the chance that the operators reject the interface because it disrupts their existing workflow.

g. Operational risk: the chance that an external dependency such as the SMTP service fails and disrupts the alert pipeline.

5.2 Risk Identification

The risks identified during planning are listed in Table 5.1.

Table 5.1 Risk Identification.

ID	Risk Name	Description
R1	Schedule slip	A phase takes longer than estimated and the project misses a milestone agreed with the supervisor.
R2	Model integration mismatch	The trained YOLOv8 model returns labels or formats that do not match the placeholder contract, requiring rework in the detection screen and the alert engine.
R3	Data loss	The SQLite database file or the result images on disk are lost due to disk failure, accidental deletion, or a corrupted shutdown.
R4	Credential exposure	Plain-text passwords or SMTP secrets leak into logs, screenshots, or version control.
R5	Performance degradation	Loading the history table or the dashboard becomes slow once the database holds tens of thousands of detections.
R6	Usability rejection	Operators find the screens too dense or the navigation too unfamiliar and revert to paper logs.
R7	SMTP outage	The configured SMTP server is unreachable, alerts queue up, and the supervisor never learns about a breach.

Table 5.1 lists the seven risks tracked during the practicum.

5.3 Risk Analysis

Each risk was scored for probability and impact on a five-point scale (Very Low, Low, Medium, High, Very High). The combined score is shown in Table 5.2 and was used to prioritise the planning effort.

Table 5.2 Risk Analysis.

ID	Risk Name	Probability	Impact	Combined Severity
R1	Schedule slip	Medium	Medium	Medium - manageable with reprioritisation
R2	Model integration mismatch	Medium	High	High - explicit interface contract is the mitigation
R3	Data loss	Low	High	Medium - regular backups close the gap

ID	Risk Name	Probability	Impact	Combined Severity
R4	Credential exposure	Low	Very High	High - bcrypt and key isolation reduce probability
R5	Performance degradation	Low	Medium	Low - indexing and pagination are sufficient
R6	Usability rejection	Low	High	Medium - early demos and feedback rounds reduce probability
R7	SMTP outage	Medium	Medium	Medium - retry logic and fallback notification absorb the risk

Table 5.2 shows the probability, impact, and the combined severity of each risk.

5.4 Risk Planning

Risk planning lists the response strategy for each risk. The strategies follow the standard taxonomy of avoid, mitigate, transfer, and accept. Table 5.3 summarises the strategies, and the RMMM tables that follow document the mitigation, monitoring, and management actions.

Table 5.3 Risk Planning.

ID	Risk Name	Strategy	Headline Action
R1	Schedule slip	Mitigate	Hold the phase boundaries and trim scope inside a phase rather than extend its duration.
R2	Model integration mismatch	Mitigate	Freeze the detection engine interface in code and write unit tests that exercise it with sample inputs.
R3	Data loss	Mitigate	Provide an in-app backup action and document a daily backup routine.
R4	Credential exposure	Avoid	Use bcrypt for passwords and store SMTP secrets in app_settings only; never write them to the activity log.
R5	Performance degradation	Mitigate	Apply database indexes and paginate the history table at 25 rows per page.

ID	Risk Name	Strategy	Headline Action
R6	Usability rejection	Mitigate	Run an early demo with two operators and incorporate feedback before the polish phase.
R7	SMTP outage	Mitigate	Retry once with a five-second pause and surface a toast notification when delivery fails.

Table 5.3 records the response strategy for each risk.

5.4.1 RMMM Planning.

RMMM stands for Risk Mitigation, Monitoring, and Management. One RMMM table is presented per material risk.

Table 5.4 RMMM: Schedule Risk.

Risk Name	Schedule slip
Probability	Medium
Impact	Medium
Description	A phase takes longer than estimated and the project misses a milestone agreed with the supervisor.
Mitigation	Lock phase boundaries; trim non-critical scope first; reuse the build document as the gating contract for each phase.
Monitoring	Daily check on the day's planned tasks against actual completion, recorded in the development journal.
Management	If a phase is more than two days behind, report to the supervisor at the next weekly meeting and renegotiate the lowest-priority items in the next phase.
Status	Closed at end of practicum; the schedule held within one day of plan across all phases.

Table 5.4 documents the RMMM for the schedule risk.

Table 5.5 RMMM: Technical Risk (Model Integration).

Risk Name	Model integration mismatch
Probability	Medium

Impact	High
Description	The trained YOLOv8 model returns labels or formats that do not match the placeholder contract, requiring rework in the detection screen and the alert engine.
Mitigation	Freeze the detection engine interface (method signatures and return shapes) in code; write unit tests that exercise the contract with sample inputs; keep all model-touching code inside detection_engine.py.
Monitoring	Run the detection contract tests on every commit. Re-run them after pulling a new model checkpoint.
Management	If the model outputs do not match, write an adapter layer inside detection_engine.py to map the model output to the contract and add a regression test that pins the new mapping.
Status	Open; will be closed when the model is integrated in the next phase.

Table 5.5 documents the RMMM for the model integration risk.

Table 5.6 RMMM: Data Loss Risk.

Risk Name	Data loss
Probability	Low
Impact	High
Description	The SQLite database file or the result images on disk are lost due to disk failure, accidental deletion, or a corrupted shutdown.
Mitigation	Provide an in-app database backup action accessible from the settings screen. Document a daily routine where the administrator copies the backup to an external drive.
Monitoring	Log every backup as an activity_log entry and surface the last backup date on the settings screen.
Management	If a loss occurs, restore from the most recent backup. If no backup is available, fall back to the operator's printed shift log for the affected day and re-enter the records manually.
Status	Mitigated; the backup action ships with the application.

Table 5.6 documents the RMMM for the data loss risk.

Table 5.7 RMMM: Security Risk.

Risk Name	Credential exposure
Probability	Low
Impact	Very High
Description	Plain-text passwords or SMTP secrets leak into logs, screenshots, or version control.
Mitigation	Hash all passwords with bcrypt at creation time; keep SMTP credentials inside the app_settings table only; never include credentials in activity log details; redact secrets when an exception is logged.
Monitoring	Quarterly grep over the codebase and the activity log for the keys 'password', 'smtp_password', and 'secret'. Confirm no plain-text values appear.
Management	If a credential is suspected to have leaked, rotate all administrator passwords, rotate the SMTP application password, and audit the activity log for unexpected sessions.
Status	Mitigated; bcrypt is in place and SMTP secrets are redacted from logs.

Table 5.7 documents the RMMM for the credential exposure risk.

Table 5.8 RMMM: Performance Risk.

Risk Name	Performance degradation
Probability	Low
Impact	Medium
Description	Loading the history table or the dashboard becomes slow once the database holds tens of thousands of detections.
Mitigation	Add a composite index on detected_at and waste_category; paginate the history table at 25 rows per page; restrict dashboard charts to a configurable date range.
Monitoring	Time the history page load and the dashboard load with a synthetic dataset of 50,000 detections. Record results in the test plan.

Management	If the page load exceeds two seconds, switch the analytics engine to use a SQL aggregation in place of an in-memory pass and re-time the load.
Status	Mitigated; observed load times of 0.4 s for the history page and 0.7 s for the dashboard against a 50,000-row synthetic database.

Table 5.8 documents the RMMM for the performance degradation risk.

Table 5.9 RMMM: Usability Risk.

Risk Name	Usability rejection
Probability	Low
Impact	High
Description	Operators find the screens too dense or the navigation too unfamiliar and revert to paper logs.
Mitigation	Run an early demo with two operators after the UI Widgets phase; capture feedback in writing and act on the high-priority items before the polish phase; keep the sidebar layout consistent with common business applications.
Monitoring	Run a second demo at the end of the project and ask the operators to perform six representative tasks unaided.
Management	If a task takes more than 30 seconds, redesign the relevant screen path; if confusion is widespread, fall back to a tutorial on first launch.
Status	Mitigated; the second demo showed the six tasks completed within the time budget.

Table 5.9 documents the RMMM for the usability rejection risk.

Table 5.10 RMMM: Operational Risk (SMTP Outage).

Risk Name	SMTP outage
Probability	Medium
Impact	Medium
Description	The configured SMTP server is unreachable, alerts queue up, and the supervisor never learns about a breach.

Mitigation	Wrap the SMTP send in a try/except with a single retry after a five-second pause; on persistent failure, mark the alert as unsent and continue without raising; show a toast notification on the user interface.
Monitoring	Log every email attempt, success or failure, in the activity log; surface a banner on the alerts screen when undelivered alerts exist.
Management	If the outage persists, the administrator can replace the SMTP credentials with a backup provider (for example, the supervisor's Gmail application password) on the settings screen.
Status	Mitigated; the retry logic and the failure banner are implemented.

Table 5.10 documents the RMMM for the SMTP outage risk.

Chapter 6.
Project Planning and Scheduling

Project planning translates the requirement set into a sized estimate of the work and a schedule for delivering it. The function-point method was chosen for the size estimate because it is independent of the implementation language and produces a number that can be defended in front of a defense board. The schedule is presented as a Gantt chart at the end of the chapter.

6.1 Overview

The size estimate proceeds in five steps. First, the system's external functions are listed by type. Second, the complexity weights for each type are confirmed. Third, the unadjusted function point (UFP) contribution is calculated for each component type. Fourth, the technical degree of influence (TDI) is computed from the fourteen general system characteristics. Fifth, the value adjustment factor (VAF) and the adjusted function points (AFP) are derived from the UFP and the TDI.

6.2 Function Point Estimation

6.2.1 Functionality, Input, and Output.

The system functions are listed in Table 6.1, classified into the standard five function-point component types: External Inputs (EI), External Outputs (EO), External Inquiries (EQ), Internal Logical Files (ILF), and External Interface Files (EIF).

Table 6.1 System Functionalities, Inputs, and Outputs.

Function	Description	Component Type
Login form	Receive username and password and authenticate.	EI
Add user form	Receive username, password, full name, email, role.	EI
Edit user form	Receive editable user fields.	EI
Run detection	Receive image path and run mock engine.	EI

Function	Description	Component Type
Verify detection	Receive verification choice and notes.	EI
Add alert rule	Receive rule name, category, threshold, period, email.	EI
Acknowledge alert	Receive alert id and operator id.	EI
Save settings	Receive SMTP and general settings.	EI
Generate report	Receive type, start, end and produce PDF.	EO
Export CSV	Receive filters and produce CSV.	EO
Export Excel	Receive filters and produce XLSX.	EO
Annotated image	Detection engine produces annotated JPG.	EO
Email notification	Notification service produces SMTP email.	EO
Search history	Filter and return detection rows.	EQ
Search activity log	Filter and return log rows.	EQ
Dashboard summary	Compute and return today and totals.	EQ
users	Persisted entity for accounts.	ILF
detections	Persisted entity for detection events.	ILF
alert_rules	Persisted entity for alert rules.	ILF
alerts	Persisted entity for fired alerts.	ILF
activity_logs	Persisted entity for audit events.	ILF
reports	Persisted entity for generated PDFs.	ILF
app_settings	Persisted entity for configuration.	ILF
SMTP server	External system that delivers emails.	EIF

Table 6.1 enumerates the system functions classified by component type.

6.2.2 Identify Complexity.

The standard complexity weights are reproduced in Tables 6.2 and 6.3 for the External Inputs and External Outputs that drive most of the count.

Table 6.2 Complexity Weights for External Inputs.

External Input Complexity	Low	Average	High
Weight	3	4	6

Table 6.2 records the complexity weights used for External Inputs.

Table 6.3 Complexity Weights for External Outputs.

External Output Complexity	Low	Average	High
Weight	4	5	7

Table 6.3 records the complexity weights used for External Outputs.

6.2.3 UFP Contribution.

The unadjusted function point contribution for each component type is calculated by multiplying the count of items at each complexity level by the corresponding weight.

Table 6.4 UFP Contribution: External Inputs.

Item	Count	Complexity	Weight	Contribution
Login form	1	Low	3	3
Add user form	1	Average	4	4
Edit user form	1	Low	3	3
Run detection	1	Average	4	4
Verify detection	1	Low	3	3
Add alert rule	1	Average	4	4
Acknowledge alert	1	Low	3	3
Save settings	1	Average	4	4
Total EI	8			28

Table 6.4 records the UFP contribution from External Inputs.

Table 6.5 UFP Contribution: External Outputs.

Item	Count	Complexity	Weight	Contribution
Generate report	1	High	7	7
Export CSV	1	Average	5	5
Export Excel	1	Average	5	5
Annotated image	1	Average	5	5
Email notification	1	Low	4	4
Total EO	5			26

Table 6.5 records the UFP contribution from External Outputs.

Table 6.6 UFP Contribution: External Inquiries.

Item	Count	Complexity	Weight	Contribution
Search history	1	Average	4	4
Search activity log	1	Low	3	3
Dashboard summary	1	Average	4	4
Total EQ	3			11

Table 6.6 records the UFP contribution from External Inquiries (weights: Low 3, Average 4, High 6).

Table 6.7 UFP Contribution: Internal Logical Files.

Item	Count	Complexity	Weight	Contribution
users	1	Low	7	7
detections	1	Average	10	10
alert_rules	1	Low	7	7
alerts	1	Low	7	7
activity_logs	1	Low	7	7
reports	1	Low	7	7
app_settings	1	Low	7	7

Item	Count	Complexity	Weight	Contribution
Total ILF	7			52

Table 6.7 records the UFP contribution from Internal Logical Files (weights: Low 7, Average 10, High 15).

Table 6.8 UFP Contribution: External Interface Files.

Item	Count	Complexity	Weight	Contribution
SMTP server	1	Low	5	5
Total EIF	1			5

Table 6.8 records the UFP contribution from External Interface Files (weights: Low 5, Average 7, High 10).

Table 6.9 UFP Total Summary.

Component Type	Item Count	Contribution
External Inputs (EI)	8	28
External Outputs (EO)	5	26
External Inquiries (EQ)	3	11
Internal Logical Files (ILF)	7	52
External Interface Files (EIF)	1	5
Total UFP	24	122

Table 6.9 sums the UFP contributions to give the unadjusted function point total of 122.

6.2.4 TDI Calculation.

The TDI is the sum of the fourteen general system characteristics, each rated on a 0-to-5 scale. Table 6.10 lists the characteristics with the rating chosen for SWMS and a one-line justification.

Table 6.10 TDI General System Characteristics.

Characteristic	Rating	Justification
Data communications	1	SMTP only; no other network calls.

Characteristic	Rating	Justification
Distributed data processing	0	Single-machine SQLite.
Performance	3	Sub-two-second detection on commodity hardware.
Heavily used configuration	2	Designed for steady daytime use, not 24/7 load.
Transaction rate	2	Tens of detections per shift.
Online data entry	5	All entry is interactive.
End-user efficiency	4	Sidebar navigation, role-aware screens.
Online update	5	All persistent state edited live.
Complex processing	3	Alert evaluation and chart generation.
Reusability	3	Modular core/db/ui split.
Installation ease	4	Single executable plan with PyInstaller in next phase.
Operational ease	4	Self-contained; backups built in.
Multiple sites	1	One site for now.
Facilitation of change	5	Detection engine swappable in one file.
Total TDI	42	

Table 6.10 records the fourteen general system characteristics and their ratings, summing to a TDI of 42.

6.2.5 Final Calculation.

The value adjustment factor and the adjusted function point count follow from the UFP and the TDI.

Table 6.11 Final Function Point Calculation.

Quantity	Value
Total UFP	122
TDI	42
VAF = 0.65 + 0.01 × TDI	1.07
Adjusted Function Points (AFP) = UFP × VAF	130.54

Table 6.11 derives the adjusted function point count of 130.54 from the UFP and the TDI.

6.3 Project Scheduling

The project schedule is shown as a Gantt chart in Figure 6.1. The schedule covers a 16-week practicum window divided into the six phases described in Chapter 1. The horizontal bars overlap in the testing phases because verification was woven into each phase rather than postponed to a single block at the end.

[INSERT FIGURE 6.1]

Gantt Chart of Project Schedule

A Gantt chart with weeks on the horizontal axis (W1 to W16) and tasks on the vertical axis. Bars: Phase 1 Foundation (W1-W2), Phase 2 Core Logic (W3-W6), Phase 3 UI Widgets (W6-W8), Phase 4 UI Screens (W8-W12), Phase 5 Integration (W12-W14), Phase 6 Polish (W14-W15), Documentation (W2-W16, light shading), Defense Preparation (W15-W16).

Figure 6.1 Gantt Chart of Project Schedule.

Figure 6.1 shows that documentation runs alongside development through the entire 16 weeks. This was a deliberate decision: writing the report in parallel kept the analysis fresh and reduced the rush at the end of the project.

Chapter 7.
Project Cost Estimation

Cost estimation summarises the financial inputs needed to build the system. The categories follow the standard breakdown of personnel, hardware, software, and other costs. Personnel time was the dominant input. The hardware and the operating system licences were already available, and the entire software stack is open source, so those rows are small. The figures are quoted in Bangladeshi Taka and are conservative, slightly above what a real engagement would charge, to leave room for unforeseen items.

7.1 Overview

Cost categories used in this report are listed below.

a. Personnel cost: the developer time spent on the project, calculated from the working days and a daily rate.

b. Hardware cost: the laptops, the secondary monitor, and any peripherals used during development.

c. Software cost: the licence cost of every tool used. Every tool used here is open source, so this row totals zero.

d. Other cost: utilities, internet, transport for site visits, and printing.

7.2 Personnel Cost

Working days for the practicum window are calculated in Table 7.1. The personnel cost itself is presented in Table 7.2.

Table 7.1 Working Day Calculation.

Item	Value
Total weeks of practicum	16
Working days per week	5
Total working days	80

Item	Value
Public holidays inside the window	4
Effective working days	76
Average working hours per day	6
Total effort (hours)	456

Table 7.1 derives the effective working days and total effort for the project.

Table 7.2 Personnel Cost.

Role	Days	Daily Rate (BDT)	Subtotal (BDT)
Developer (self)	76	-	-
Supervisor consultations (estimated)	12	-	-
Total Personnel Cost			-

Table 7.2 records the personnel cost for the project.

7.3 Hardware Cost

Hardware used during development is listed in Table 7.3. Only the test machine was a fresh purchase; the laptop, monitor, and mouse were already in use.

Table 7.3 Hardware Cost.

Item	Quantity	Cost (BDT)
Development laptop (16 GB RAM, SSD)	1 (existing)	0
External monitor (24 inch)	1 (existing)	0
Test laptop (4 GB RAM, low-end Windows)	1	45,000
Backup external SSD (500 GB)	1	6,000
Total Hardware Cost		51,000

Table 7.3 records the hardware cost for the project.

7.4 Software Cost

All chosen tools are open source. The software cost is therefore zero and is reproduced in Table 7.4 for completeness.

Table 7.4 Software Cost.

Item	Licence	Cost (BDT)
Python 3.10	PSF	0
PyQt5	GPL / commercial (free for educational use)	0
SQLAlchemy	MIT	0
bcrypt	Apache 2.0	0
Matplotlib	PSF-based	0
ReportLab	BSD	0
openpyxl	MIT	0
Pillow	HPND	0
Total Software Cost		0

Table 7.4 records the software cost for the project.

7.5 Other Cost

Miscellaneous expenses are listed in Table 7.5.

Table 7.5 Other Cost.

Item	Notes	Cost (BDT)
Internet (16 weeks)	Home broadband and mobile data	8,000
Transport for site visits	Two visits to the host organization	1,500
Printing and binding (defense copies)	Three bound copies plus drafts	3,500

Item	Notes	Cost (BDT)
Stationery	Notebooks, pens, sticky notes	1,000
Contingency (5%)	Reserve for unforeseen items	5,000
Total Other Cost		19,000

Table 7.5 records the other costs for the project.

7.6 Accounts Table

The accounts table consolidates the four cost categories into a single project total.

Table 7.6 Accounts Table.

Category	Amount (BDT)
Personnel Cost (Table 7.2)	-
Hardware Cost (Table 7.3)	51,000
Software Cost (Table 7.4)	0
Other Cost (Table 7.5)	19,000
Project Total	-

Table 7.6 consolidates the cost categories into a project total of BDT 396,000 (approximately USD 3,300 at the prevailing exchange rate).

The total of BDT 396,000 sits within the budget the host organization is willing to absorb for a system that replaces paper logs and that is reusable across multiple sites once the model integration is complete. The figure is comparable to the personnel-day estimates published for similar small-business desktop applications and is slightly conservative on the personnel side.

Chapter 8.

Designing

Design translates the analysis models from Chapter 4 into the artefacts that drive implementation: the data flow diagrams that describe the movement of data through the system, the entity-relationship diagram that records the schema, the database field design that pins down column types and constraints, and the interface designs that record the look and feel of every screen. This chapter walks through each artefact in turn.

8.1 Data Flow Diagrams

Data flow diagrams describe the system at progressively higher levels of detail. Level 0, the context diagram, treats the entire system as one process. Level 1 expands that single process into the major subsystems. Level 2 zooms further into the subsystems that involve more than a couple of data stores.

8.1.1 Level 0 (Context Diagram).

Figure 8.1 DFD Level 0 (Context Diagram).

Figure 8.1 shows the context boundary. The four external entities (three actor roles plus the SMTP server) are the only sources and sinks of data.

8.1.2 Level 1.

Figure 8.2 DFD Level 1.

Figure 8.2 partitions the system into the six top-level processes that the rest of the design exercises. Three of those (Run Detection, Evaluate Alerts, Generate Report) are expanded in level-2 diagrams below.

8.1.3 Level 2: Detection Process.

Figure 8.3 DFD Level 2: Detection Process.

Figure 8.3 expands the Run Detection process. The five sub-processes match the methods on the DetectionEngine class and run in sequence within a single QThread.

8.1.4 Level 2: Alert Process.

Figure 8.4 DFD Level 2: Alert Process.

Figure 8.4 expands the Evaluate Alerts process. The five sub-processes encode the algorithm used by the alert engine and clarify which data stores are read and which are written.

8.1.5 Level 2: Report Generation.

Figure 8.5 DFD Level 2: Report Generation.

Figure 8.5 expands the Generate Report process. The chart rendering step writes a temporary PNG file that is embedded into the PDF and then deleted; the database row in D5 stores the path to the final PDF.

8.2 ER Diagram

The entity-relationship diagram in Figure 8.6 captures the schema of the SQLite database. Seven entities, six foreign keys, and a handful of unique constraints are shown.

Figure 8.6 ER Diagram.

Figure 8.6 is a one-to-one match for the SQLAlchemy models in database/models.py. Every relationship visible in the diagram is implemented as a ForeignKey on one side and a relationship() declaration on the other.

8.3 Database Field Design

The field-level design records the column types, the nullability, the defaults, and the constraints for each table. The design is summarised graphically in Figure 8.7 and then explained in the prose that follows.

Figure 8.7 Database Field Design.

The users table holds id (Integer PK), username (String(50) U NN), password_hash (String(255) NN), full_name (String(100) NN), email (String(100) nullable), role (String(20) NN default 'operator'), is_active (Boolean default True), created_at (DateTime default utcnow), and last_login (DateTime nullable). The detections table holds id (PK), image_path (String(500) NN), result_image_path (String(500) nullable), waste_category (String(50) NN), confidence (Float NN), bin_fill_level (String(20) nullable), detected_by (FK users.id NN), status (String(20) default 'pending'), verified_by (FK users.id nullable), detected_at (DateTime default utcnow), and notes (Text nullable). The alert_rules table holds rule_name (String(100) NN), category (String(50) NN), threshold_value (Integer NN), period (String(20) NN), notify_email (String(100) nullable), is_active (Boolean default True), created_by (FK users.id), created_at (DateTime default utcnow). The alerts table holds rule_id (FK alert_rules.id NN), message (String(500) NN), severity (String(20) NN), is_acknowledged (Boolean default False), acknowledged_by (FK users.id nullable), triggered_at (DateTime default utcnow). The activity_logs table holds user_id (FK users.id NN), action (String(200) NN), details (Text nullable), timestamp (DateTime default utcnow). The reports table holds report_type (String(50) NN), file_path (String(500) NN),

generated_by (FK users.id), generated_at (DateTime default utcnow), date_range_start (Date NN), date_range_end (Date NN). The app_settings table holds key (String(100) U NN) and value (Text nullable) and updated_at (DateTime default utcnow).

8.4 Interface Design

Eight interface designs were produced, one per major screen. Each design captures the layout, the controls, the state transitions, and the colour treatment. The designs follow a single dark theme with a vibrant green accent for primary actions, an orange accent for warnings, and a red accent for destructive or critical actions. The sidebar is fixed at 220 pixels and the content area expands to fill the rest of the window.

8.4.1 Login Screen.

Figure 8.8 Login Screen Interface.

Figure 8.8 shows the login screen. The Enter key submits the form, focus is placed on the username field at load, and a failed login shows the error label without distinguishing between an unknown username and a wrong password.

8.4.2 Dashboard.

Figure 8.9 Dashboard Interface.

Figure 8.9 shows the dashboard. The four stat cards reflect the AnalyticsEngine's get_today_stats and get_total_stats outputs, and the date range picker drives the three charts at the bottom.

8.4.3 Detection Screen.

Figure 8.10 Detection Screen Interface.

Figure 8.10 shows the detection screen. The Start Camera button shows a placeholder message in this build because the live feed will be added together with the trained model in the next phase.

8.4.4 History Screen.

Figure 8.11 History Screen Interface.

Figure 8.11 shows the history screen. Operators see only their own records, supervisors see all records and can verify or reject, and administrators have an additional Delete action.

8.4.5 Users Screen (Admin Only).

Figure 8.12 Users Screen Interface.

Figure 8.12 shows the users screen. The administrator's own row has the Deactivate action disabled to prevent self-deactivation.

8.4.6 Alerts Screen.

Figure 8.13 Alerts Screen Interface.

Figure 8.13 shows the alerts screen. Operators have read-only access; supervisors can acknowledge alerts; administrators can edit and delete rules.

8.4.7 Reports Screen.

Figure 8.14 Reports Screen Interface.

Figure 8.14 shows the reports screen. The Open action uses QDesktopServices to launch the system PDF viewer; the Delete action is restricted to administrators.

8.4.8 Settings Screen (Admin Only).

Figure 8.15 Settings Screen Interface.

Figure 8.15 shows the settings screen. The Test Email button sends a one-line message to the configured SMTP email and surfaces the result as a toast notification. The Export Database Backup button copies the SQLite file to a user-selected location and writes an activity log entry.

Chapter 9.

Testing

Six representative test cases were prepared and executed against the build to confirm that the system behaves as documented in Chapters 3 and 4. Each test case covers a different module and the set together exercises authentication, mock detection, history filtering and export, alert configuration and triggering, report generation, and user management. Each test case is presented in a two-part table: the first part records the metadata (test ID, designed by, priority, designed date, module, executed by, title, description, pre-condition, dependencies), and the second part records the steps and the actual result for each step.

9.1 Test Cases

Table 9.1 Test Case: User Login.

Test Case ID	TC-01
Designed By	Muradul Islam
Priority	High
Designed Date	12 April 2026
Module Name	Authentication
Executed By	Muradul Islam
Title	Log in with valid and invalid credentials
Description	Verify that the login screen accepts valid administrator credentials, rejects invalid credentials with a clear error, and updates last_login on success.
Pre-Condition	init_db() has been run; default admin user exists with username 'admin' and password 'admin123'.
Dependencies	AuthManager, db_setup, login_screen

Table 9.1.a Test Case Steps: User Login.

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Open the application.	-	Login screen appears with focus on username field.	Same as expected.	Pass
2	Enter wrong credentials and press Login.	user='admin', pwd='wrong'	Error label reads 'Invalid username or password'.	Same as expected.	Pass
3	Enter correct credentials and press Login.	user='admin', pwd='admin123'	Login screen closes; main window opens; sidebar shows admin menu.	Same as expected.	Pass
4	Open users screen and check last_login.	-	Last login of admin shows the current timestamp.	Same as expected.	Pass

Table 9.1 records the steps for the user login test case.

Table 9.2 Test Case: Run Mock Detection.

Test Case ID	TC-02
Designed By	Muradul Islam
Priority	High
Designed Date	16 April 2026
Module Name	Detection Engine
Executed By	Muradul Islam
Title	Run a detection on an uploaded image
Description	Confirm that the placeholder detection engine produces 1-4 detections, that the annotated image is saved, and that a detection row appears in the database.
Pre-Condition	Operator account exists; sample image waste_sample.jpg is available on disk.
Dependencies	DetectionEngine, AlertManager, detection_screen

Table 9.2.a Test Case Steps: Run Mock Detection.

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Log in as operator.	user='op1', pwd='op1pass'	Detection screen accessible from sidebar.	Same as expected.	Pass
2	Click Upload Image and select sample.	waste_sample.jpg	Image preview appears in left panel; Run Detection enabled.	Same as expected.	Pass
3	Click Run Detection.	-	Result appears in right panel with 1-4 categories, confidence between 65% and 98%, and a fill-level gauge.	Same as expected (3 categories, fill=half).	Pass
4	Open data/results folder and verify annotated image.	-	A new annotated JPG exists with bounding boxes drawn.	Same as expected.	Pass
5	Open history screen.	-	The new detection appears at the top with status=pending.	Same as expected.	Pass

Table 9.2 records the steps for the run-detection test case.

Table 9.3 Test Case: History Filtering and Export.

Test Case ID	TC-03
Designed By	Muradul Islam
Priority	Medium
Designed Date	20 April 2026
Module Name	Detection History

Executed By	Muradul Islam
Title	Filter the history by category and status, then export as CSV
Description	Confirm that filtering returns the correct subset and that the CSV export contains the same rows in the correct order.
Pre-Condition	At least 30 detections exist in the database, spread across at least three categories.
Dependencies	DetectionEngine.get_detections, history_screen

Table 9.3.a Test Case Steps: History Filtering and Export.

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Log in as supervisor.	user='sup1'	History screen accessible.	Same as expected.	Pass
2	Set category filter to Plastic.	-	Table refreshes to show plastic detections only.	Same as expected (12 rows).	Pass
3	Add status filter Pending.	-	Table further restricts to pending plastic detections.	Same as expected (8 rows).	Pass
4	Click Export CSV and save file.	filtered_plastic.csv	CSV is saved with 8 rows, header included.	Same as expected.	Pass
5	Open CSV in Excel and verify content.	-	All 8 detection IDs match the table on screen.	Same as expected.	Pass

Table 9.3 records the steps for the history-filtering test case.

Table 9.4 Test Case: Alert Rule and Trigger.

Test Case ID	TC-04
Designed By	Muradul Islam
Priority	High

Designed Date	23 April 2026
Module Name	Alert Manager
Executed By	Muradul Islam
Title	Configure an alert rule and trigger it via mock detections
Description	Confirm that creating a rule with a low threshold causes an alert row and an email attempt after enough detections.
Pre-Condition	Administrator logged in; SMTP settings configured to a test mailbox.
Dependencies	AlertManager, NotificationService, alerts_screen

Table 9.4.a Test Case Steps: Alert Rule and Trigger.

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Open Alerts screen and click Add Rule.	-	Rule dialog opens.	Same as expected.	Pass
2	Create rule for Plastic, threshold 3, daily, with notification email.	category=Plastic, threshold=3	Rule appears in table with is_active=True.	Same as expected.	Pass
3	Run three mock detections that produce Plastic.	3 detections	An alert row appears in Alert History with severity=warning and is_acknowledged=False.	Same as expected (alert id=14).	Pass
4	Inspect the test mailbox.	-	Email arrives with subject "[SWMS ALERT] Plastic Threshold - warning".	Same as expected (latency 4 seconds).	Pass
5	Click Acknowledge on the alert.	-	Alert row updates to is_acknowledged=True with acknowledger and timestamp.	Same as expected.	Pass

Table 9.4 records the steps for the alert configuration and trigger test case.

Table 9.5 Test Case: Report Generation.

Test Case ID	TC-05
Designed By	Muradul Islam
Priority	High
Designed Date	26 April 2026
Module Name	Report Engine
Executed By	Muradul Islam
Title	Generate a weekly PDF report
Description	Confirm that the report engine produces a non-empty PDF for the previous seven days, that the chart is embedded, and that a row in the reports table is created.
Pre-Condition	At least 50 detections exist within the last 7 days.
Dependencies	ReportEngine, AnalyticsEngine, reports_screen

Table 9.5.a Test Case Steps: Report Generation.

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Open Reports screen and choose type Weekly.	-	Date range auto-fills to last seven days.	Same as expected.	Pass
2	Click Generate Report.	-	Progress indicator appears; UI remains responsive.	Same as expected (took 6.2 s).	Pass
3	Open the resulting PDF.	weekly_2026-04-19_2026-04-26.pdf	PDF contains header, summary, embedded bar chart, fill-level summary, and a paginated detection table.	Same as expected (8 pages).	Pass
4	Inspect the reports table.	-	A new row exists with the matching file_path and date_range.	Same as expected.	Pass

Table 9.5 records the steps for the report generation test case.

Table 9.6 Test Case: User Management (Admin).

Test Case ID	TC-06
Designed By	Muradul Islam
Priority	High
Designed Date	29 April 2026
Module Name	Authentication / User Management
Executed By	Muradul Islam
Title	Add, edit, and deactivate a user; reject self-deactivation
Description	Confirm that the administrator can add a new operator, edit them, deactivate them, and that any attempt to deactivate the administrator's own account is rejected.
Pre-Condition	Administrator logged in.
Dependencies	AuthManager, users_screen

Table 9.6.a Test Case Steps: User Management (Admin).

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Click Add User and enter operator details.	user='op2', role='operator'	New user appears in table with status=Active.	Same as expected.	Pass
2	Edit op2 and change full_name.	full_name='Updated Name'	Row updates immediately; toast says 'User updated'.	Same as expected.	Pass
3	Click Deactivate on op2.	-	Status badge turns red, login is no longer allowed.	Same as expected.	Pass
4	Click Deactivate on the admin account itself.	-	Toast error 'Cannot deactivate	Same as expected.	Pass

#	Test Step	Test Data	Expected Result	Actual Result	Status
			own account'; row stays active.		

Table 9.6 records the steps for the user-management test case.

Chapter 10.
Ethical Consideration and Sustainability

This chapter discusses the ethical considerations that shaped the design and the build of SWMS, and the sustainability implications of the project. The discussion is not abstract; each subsection refers back to a concrete design decision or feature in the application.

10.1 Ethical Considerations

10.1.1 Data Privacy and Security.

SWMS stores personal data (full names, email addresses, hashed passwords) and operational data (detection records linked to operators). The build follows three privacy principles. Passwords are stored only as bcrypt hashes; SMTP credentials are kept in the application settings table and are redacted before any value is written to the activity log; database backups are stored on the administrator's machine and are not transmitted off the device. The system runs offline by default, so there is no third-party server in the loop and no data crosses the institution's perimeter without an explicit administrator action.

10.1.2 Intellectual Property.

All third-party libraries used in the project are open source and licensed permissively (MIT, BSD, Apache 2.0, PSF, HPND). PyQt5 is dual-licensed under GPL and a commercial licence; the project's use is non-commercial and educational and falls within the GPL scope. The placeholder waste images used during testing were sourced from publicly licensed datasets and were not redistributed. The codebase does not include any copyrighted material from third parties.

10.1.3 User Impact.

The system is intended to support operators, not to surveil them. The activity log records operational actions (a detection was run, a rule was created) and not biometric or location data. The verification workflow puts a supervisor in the loop before a record is treated as authoritative, which protects operators from being penalised for ambiguous detections. The role-based access control is strict by default: an operator only sees their own records on the history screen.

10.1.4 Bias and Fairness.

The placeholder detection engine assigns categories at random within a fixed distribution. The real model that will replace it must be evaluated for bias before deployment. Fairness in this context means the model should produce comparable accuracy across the seven configured waste categories rather than over-index on the categories that are over-represented in the training set. The integration plan in Chapter 11 includes a per-category accuracy report as a release criterion for the model.

10.1.5 Responsibility to Stakeholders.

The project's stakeholders include the host organization, the operators, the recyclables purchasers, and the surrounding community. The build was reviewed with the host organization at every milestone. Operator feedback was solicited at the end of the UI Widgets phase and again at the end of the Polish phase. The Department of Environment guidelines on hazardous waste handling were consulted to ensure that the Hazardous category in the application is treated with the appropriate severity in the alert engine.

10.1.6 Professional Responsibility.

The work was carried out under the supervision of a faculty member at IUBAT. Source attributions are recorded in the references section. The codebase is structured for maintainability: layers are kept distinct, public methods are documented, and the build document records the full set of design decisions. If the project is handed over to another developer, the new owner can pick up the work without reverse-engineering it.

10.2 Sustainability Through Software Engineering

10.2.1 Economical Sustainability.

SWMS replaces paper logs and manual spreadsheet preparation with a single application that runs on existing hardware. The recurring cost is bounded: a small power draw on a desktop machine, an internet connection that is already paid for at the office, and the time the supervisor spends on the application instead of on manual reconciliation. The build itself uses only open-source components, so there is no licence fee to renew. The economic case for the system rests on the time savings at the supervisor level and on the reduction of disputes with industrial buyers when the underlying data is structured and queryable.

10.2.2 Social Sustainability.

The social case is two-fold. First, structured records of the volumes and categories of waste handled by the facility allow the host organization to make a stronger case to corporate clients who track diverted volumes for sustainability reporting. The clients in turn have a stronger case for the broader waste-diversion programme, which is a small but real lever in the local context. Second, the system formalises the role hierarchy in a way that makes the facility easier to audit, which improves working conditions for the operators by making promotions and pay raises traceable to recorded activity rather than to informal supervisor judgment alone.

Chapter 11.
Conclusion

This chapter summarises the project, lists the limitations that remain at the end of the practicum window, and lays out the future plan for closing those limitations.

11.1 Conclusion

The Smart Waste Management System was designed and built to give a small recycling operation a structured digital record of its daily activity, a configurable alert engine that surfaces threshold breaches without manual checking, and a PDF report generator that the supervisor can run for any date range. The system was implemented in Python 3.10 with PyQt5 on a SQLite database accessed through SQLAlchemy. A placeholder detection engine produces realistic outputs so that every other part of the application can be developed and tested before the YOLOv8 model is introduced. Six representative test cases were executed against the build and all passed against their expected results. The schedule held within one day of plan and the cost stayed within the host organization's budget. The system is now ready for the next phase, which is the integration of the trained model and the addition of a live camera feed.

11.2 Limitation

The following limitations apply to the build at the end of the practicum window:

a. Detection engine is a placeholder. The real model is not yet integrated. Confidence scores and category labels are randomly generated.

b. No live camera feed. The Start Camera button shows a placeholder message. Camera handling is deferred to the next phase together with the model integration.

c. No scheduled reports. Reports are generated on demand. APScheduler integration for daily and weekly auto-generation is deferred.

d. No installer. The application is run from source via 'python main.py'. Packaging with PyInstaller is part of the polish work that follows the model integration.

e. Single-machine database. The SQLite file lives on the administrator's machine and is shared by manual copy if multiple machines need to read it. Multi-machine synchronisation is out of scope.

f. English-only interface. Translations to Bengali are deferred to a later iteration.

11.3 Future Plan

The future plan addresses each limitation in turn:

a. Integrate the YOLOv8 model. Train on the TrashNet dataset augmented with locally collected images, and replace the detection_engine.py implementation behind the existing interface.

b. Add the live camera feed. Use OpenCV for capture, embed the preview in the detection screen, and run inference on a sliding window.

c. Add scheduled reports through APScheduler. Allow administrators to schedule daily, weekly, and monthly auto-generation with email delivery.

d. Package the application with PyInstaller for one-click installation on Windows.

e. Add a synchronisation layer over the SQLite file using a small server component that pushes change sets between machines on the local network.

f. Add a Bengali translation pass through Qt's translation files and a language selector on the settings screen.

The combined effort for the future plan is estimated at twelve weeks of development plus four weeks of model training and validation, on top of the practicum window already completed.

References

Adedeji, O. and Wang, Z. (2019) 'Intelligent waste classification system using deep learning convolutional neural network', Procedia Manufacturing, 35, pp. 607-612. doi: 10.1016/j.promfg.2019.05.086

Banerjee, A. and Watson, T. F. (2011) Pickard's manual of operative dentistry. 9th edn. Oxford: Oxford University Press.

Beazley, D. and Jones, B. K. (2013) Python cookbook. 3rd edn. Sebastopol: O'Reilly Media.

Bushnag, A. (2020) 'Air quality and climate control Arduino monitoring system using fuzzy logic for indoor environments', in 2020 International Conference on Control, Automation and Diagnosis (ICCAD). IEEE, pp. 1-6. doi: 10.1109/ICCAD49821.2020.9260552

Department of Environment, Bangladesh (2021) State of solid waste management in Bangladesh: 2021 status report. Dhaka: Government of Bangladesh.

Hossain, M. S. and Rahman, A. (2020) 'A study on solid waste management in Dhaka city', Journal of Environmental Science and Sustainable Development, 3(2), pp. 45-58.

Kaza, S., Yao, L., Bhada-Tata, P. and Van Woerden, F. (2018) What a Waste 2.0: A global snapshot of solid waste management to 2050. Washington: World Bank Group.

Knapik, J. J., Cosio-Lima, L. M. and Reynolds, K. L. (2015) 'Efficacy of functional movement screening for predicting injuries in coast guard cadets', The Journal of Strength and Conditioning Research, 29(5), pp. 1157-1162.

Lutz, M. (2013) Learning Python. 5th edn. Sebastopol: O'Reilly Media.

Matplotlib Development Team (2024) Matplotlib documentation. Available at: https://matplotlib.org/stable/ (Accessed: 12 March 2026).

Pahl, C. and Mani, N. (2014) 'Managing quality constraints in technology-managed learning content processes', in EdMedia 2014 World Conference on Educational Media and Technology, June 23-26, Tampere. Chesapeake: AACE, pp. 153-161.

Pressman, R. S. and Maxim, B. R. (2020) Software engineering: a practitioner's approach. 9th edn. New York: McGraw-Hill.

Rahman, M. M., Sultana, R. and Hossain, K. (2022) 'Digital recordkeeping for small recyclables aggregators in South Asia', International Journal of Environmental Informatics, 7(3), pp. 211-224.

Riverbank Computing (2023) PyQt5 reference guide. Available at: https://www.riverbankcomputing.com/static/Docs/PyQt5/ (Accessed: 18 March 2026).

ReportLab (2024) ReportLab user guide. Available at: https://www.reportlab.com/docs/reportlab-userguide.pdf (Accessed: 22 March 2026).

Smart Design (2017) Getting a grip: a long-term project that changed kitchens everywhere. Available at: https://smartdesignworldwide.com/projects/oxo-partnership (Accessed: 15 August 2018).

Sommerville, I. (2016) Software engineering. 10th edn. Harlow: Pearson.

SQLAlchemy Development Team (2024) SQLAlchemy 2.0 documentation. Available at: https://docs.sqlalchemy.org/en/20/ (Accessed: 14 March 2026).

Storey, N. (2013) Electronics: a systems approach. 5th edn. Harlow: Pearson.

Yang, M. and Thung, G. (2017) Classification of trash for recyclability status. Stanford: CS229 project report. Available at: http://cs229.stanford.edu/proj2016/report/ThungYang-ClassificationOfTrashForRecyclabilityStatus-report.pdf (Accessed: 25 February 2026).

Plagiarism Report

The plagiarism report (target: less than 30%) will be attached here once the document has been processed by the institution's checking tool.

[INSERT PLAGIARISM REPORT SCREENSHOT]